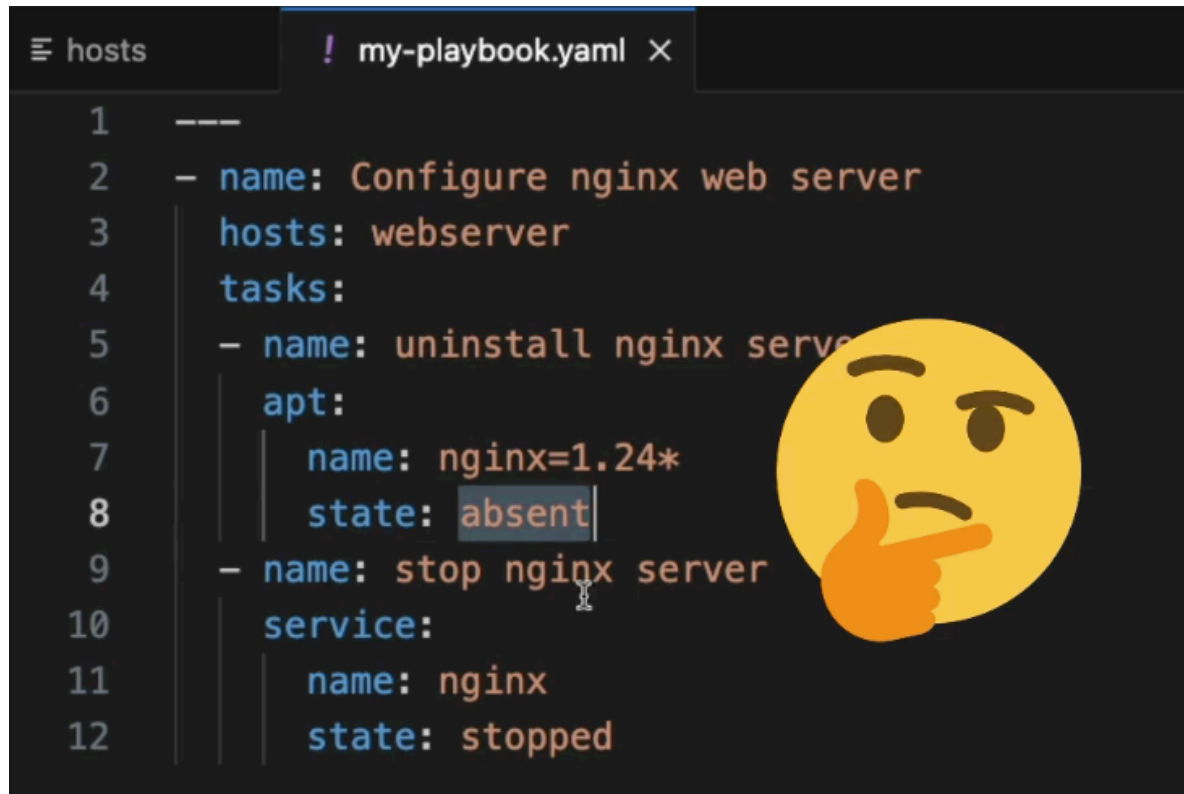


Module 15 - chapter 8

Modules & Collections in Ansible

How do we know the attribute names (keys) that we used in the playbook yaml and how do we know the values we set for those attributes?

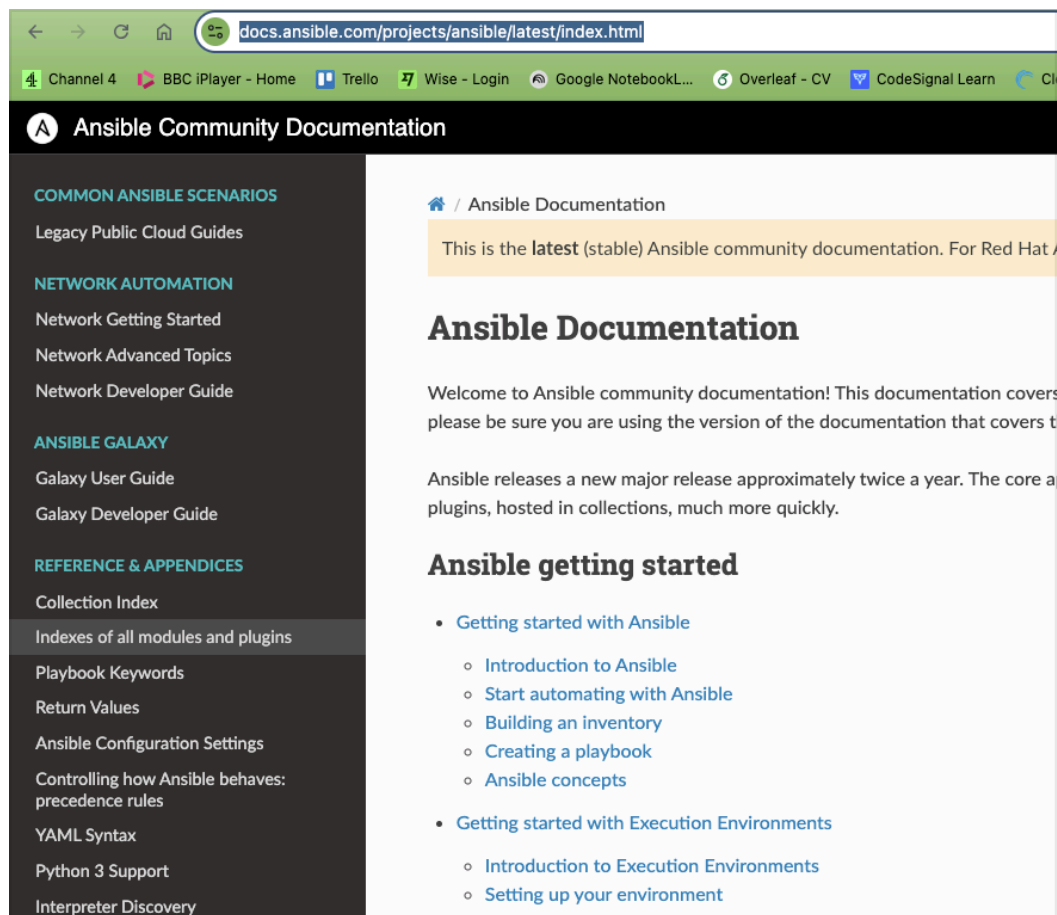


```
1 ---
2 - name: Configure nginx web server
3   hosts: webserver
4   tasks:
5     - name: uninstall nginx server
6       apt:
7         name: nginx=1.24*
8         state: absent
9     - name: stop nginx server
10      service:
11        name: nginx
12        state: stopped
```

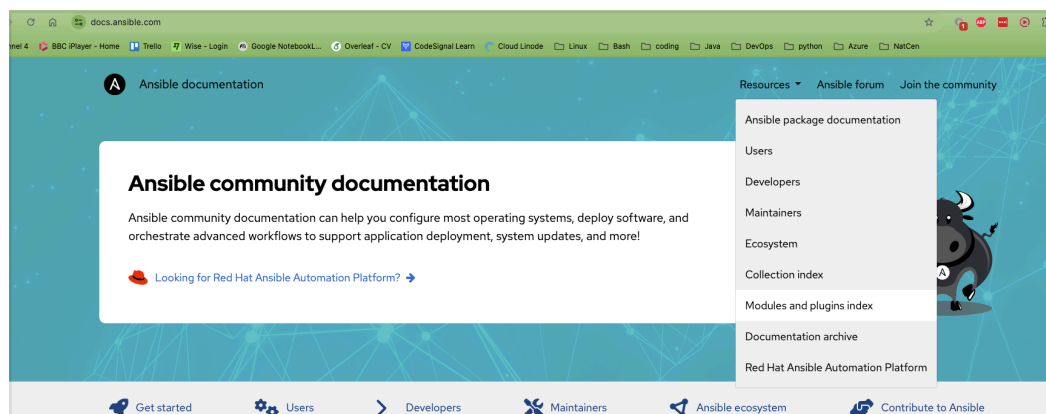
Answer -> using Ansible modules which are part of Ansible Collections

So a module is a reusable piece of code that we can use in our playbook file -> instead of writing this code ourselves we just use an existing one.

And Ansible has lots of modules for different use cases and we can find this information in Ansible documentation <https://docs.ansible.com/projects/ansible/latest/index.html>



Or here <https://docs.ansible.com/>



And then we select Modules and plugins index it will take us here: https://docs.ansible.com/projects/ansible/latest/collections/all_plugins.html

docs.ansible.com/projects/ansible/latest/collections/all_plugins.html

Ansible Community Documentation

Network Developer Guide

ANSIBLE GALAXY

Galaxy User Guide

Galaxy Developer Guide

REFERENCE & APPENDICES

Collection Index

Indexes of all modules and plugins

Index of all Become Plugins

Index of all Cache Plugins

Index of all Callback Plugins

Index of all Cliconf Plugins

Index of all Connection Plugins

Index of all Filter Plugins

Index of all Httpapi Plugins

Index of all Inventory Plugins

Index of all Lookup Plugins

Index of all Modules

Index of all Netconf Plugins

Index of all Roles

Index of all Shell Plugins

Index of all Strategy Plugins

Index of all Test Plugins

Index of all Vars Plugins

Index of all deprecated collections and plugins

Playbook Keywords

Return Values

Ansible Configuration Settings

Indexes of all modules and plugins

This is the latest (stable) Ansible community documentation. For Red Hat Ansible Automation Platform subscribers

Indexes of all modules and plugins

Plugin indexes

- Index of all Become Plugins
- Index of all Cache Plugins
- Index of all Callback Plugins
- Index of all Cliconf Plugins
- Index of all Connection Plugins
- Index of all Filter Plugins
- Index of all Httpapi Plugins
- Index of all Inventory Plugins
- Index of all Lookup Plugins
- Index of all Modules
- Index of all Netconf Plugins
- Index of all Roles
- Index of all Shell Plugins
- Index of all Strategy Plugins
- Index of all Test Plugins
- Index of all Vars Plugins
- Index of all deprecated collections and plugins

Previous

© Copyright Ansible project contributors. Last updated on Jul 14, 2026.

And we can see the list of modules

docs.ansible.com/projects/ansible/latest/collections/index_module.html

Ansible Community Documentation

ANSIBLE GALAXY

Galaxy User Guide

Galaxy Developer Guide

REFERENCE & APPENDICES

Collection Index

Indexes of all modules and plugins

Index of all Become Plugins

Index of all Cache Plugins

Index of all Callback Plugins

Index of all Cliconf Plugins

Index of all Connection Plugins

Index of all Filter Plugins

Index of all Httpapi Plugins

Index of all Inventory Plugins

Index of all Lookup Plugins

Index of all Modules

amazon.aws

- amazon.aws.autoscaling_group - Create or delete AWS AutoScaling Groups (ASGs)
- amazon.aws.autoscaling_group_info - Gather information about EC2 Auto Scaling Groups (ASGs) in AWS
- amazon.aws.autoscaling_instance - manage instances associated with AWS AutoScaling Groups (ASGs)
- amazon.aws.autoscaling_instance_info - describe instances associated with AWS AutoScaling Groups (ASGs)
- amazon.aws.autoscaling_instance_refresh - Start or cancel an EC2 Auto Scaling Group (ASG) instance refresh in AWS
- amazon.aws.autoscaling_instance_refresh_info - Gather information about EC2 Auto Scaling Group (ASG) instance refreshes in AWS
- amazon.aws.aws_az_info - Gather information about availability zones in AWS
- amazon.aws.aws_caller_info - Get information about the user and account being used to make AWS calls
- amazon.aws.aws_region_info - Gather information about AWS regions
- amazon.aws.backup_plan - Manage AWS Backup Plans
- amazon.aws.backup_plan_info - Describe AWS Backup Plans
- amazon.aws.backup_restore_job_info - List information about backup restore jobs
- amazon.aws.backup_selection - Create, delete and modify AWS Backup selection
- amazon.aws.backup_selection_info - Describe AWS Backup Selections
- amazon.aws.backup_tag - Manage tags on backup plan, backup vault, recovery point
- amazon.aws.backup_tag_info - List tags on AWS Backup resources
- amazon.aws.backup_vault - Manage AWS Backup Vaults
- amazon.aws.backup_vault_info - Describe AWS Backup Vaults
- amazon.aws.cloudformation - Create or delete an AWS CloudFormation stack
- amazon.aws.cloudformation_info - Obtain information about an AWS CloudFormation stack
- amazon.aws.cloudtrail - manage CloudTrail create, delete, update
- amazon.aws.cloudtrail_info - Gather information about trails in AWS Cloud Trail
- amazon.aws.cloudwatch_metric_alarm - Create/update or delete AWS CloudWatch 'metric alarms'
- amazon.aws.cloudwatch_metric_alarm_info - Gather information about the alarms for the specified metric
- amazon.aws.cloudwatchevent_rule - Manage CloudWatch Event rules and targets
- amazon.aws.cloudwatchlogs_log_group - create or delete log_group in CloudWatchLogs
- amazon.aws.cloudwatchlogs_log_group_info - Get information about log_group in CloudWatchLogs
- amazon.aws.cloudwatchlogs_log_group_metric_filter - Manage CloudWatch log group metric filter

Each line above is a module 🙏

Let's look at mondodb user

Ansible Community Documentation

community.mongodb

community.okd

community.postgresql

community.proxmox

community.proxysql

community.rabbitmq

community.routeros

community.sap_libs

community.sops

community.vmware

community.windows

community.zabbix

containers.podman

cyberark.pas

dellemc.enterprise_sonic

dellemc.openmanage

dellemc.powerflex

dellemc.unify

f5networks.f5_modules

fortinet.fortimanager

fortinet.fortios

google.cloud

grafana.grafana

graphiant.naas

hetzner.hcloud

community.mongodb

- community.mongodb.mongodb_atlas_cluster – Manage database clusters in Atlas
- community.mongodb.mongodb_atlas_federation_role_mapping – Manage Atlas Federation role mappings for SAML/OIDC groups
- community.mongodb.mongodb_atlas_ldap_user – Manage LDAP users in Atlas
- community.mongodb.mongodb_atlas_oidc_user – Manage OIDC database users in Atlas
- community.mongodb.mongodb_atlas_org_invite – Manage Atlas organization user invitations
- community.mongodb.mongodb_atlas_user – Manage database users in Atlas
- community.mongodb.mongodb_atlas_whitelist – Manage IP whitelists in Atlas
- community.mongodb.mongodb_balancer – Manages the MongoDB Sharded Cluster Balancer.
- community.mongodb.mongodb_index – Creates or drops indexes on MongoDB collections.
- community.mongodb.mongodb_info – Gather information about MongoDB instance.
- community.mongodb.mongodb_maintenance – Enables or disables maintenance mode for a secondary member.
- community.mongodb.mongodb_oplog – Resizes the MongoDB oplog.
- community.mongodb.mongodb_parameter – Change an administrative parameter on a MongoDB server
- community.mongodb.mongodb_replicaset – Initialises a MongoDB replicaset.
- community.mongodb.mongodb_role – Adds or removes a role from a MongoDB database
- community.mongodb.mongodb_schema – Manages MongoDB Document Schema Validators.
- community.mongodb.mongodb_shard – Add or remove shards from a MongoDB Cluster
- community.mongodb.mongodb_shard_tag – Manage Shard Tags.
- community.mongodb.mongodb_shard_zone – Manage Shard Zones.
- community.mongodb.mongodb_shell – Run commands via the MongoDB shell.
- community.mongodb.mongodb_shutdown – Cleans up all database resources and then terminates the mongod/mongos process.
- community.mongodb.mongodb_status – Validates the status of the replicaset.
- community.mongodb.mongodb_stepdown – Step down the MongoDB node from a PRIMARY state.
- community.mongodb.mongodb_user – Adds or removes a user from a MongoDB database

community.okd

Then we click inside and we can see the documentation of how to use the module, what are the parameters, what are the values and how to use the parameters:

Ansible Community Documentation

Ansible

Search docs

ANSIBLE GETTING STARTED

Getting started with Ansible

Getting started with Execution Environments

INSTALLATION, UPGRADE & CONFIGURATION

Installation Guide

Ansible Porting Guides

USING ANSIBLE

Building Ansible inventories

Using Ansible command line tools

Using Ansible playbooks

Protecting sensitive data with Ansible vault

Using Ansible modules and plugins

Using Ansible collections

Using Ansible on Windows, BSD, and z/OS UNIX

Ansible tips and tricks

CONTRIBUTING TO ANSIBLE

Ansible Community Guide

Ansible Collections Contributor Guide

ansible-core Contributors Guide

Advanced Contributor Guide

Ansible documentation style guide

EXTENDING ANSIBLE

Developer Guide

community.mongodb.mongodb_user module – Adds or removes a user from a MongoDB database

Note

This module is part of the `community.mongodb` collection (version 1.8.0).

You might already have this collection installed if you are using the `ansible` package. It is not included in `ansible-core`. To check whether it is installed, run `ansible-galaxy collection list`.

To install it, use: `ansible-galaxy collection install community.mongodb`. You need further requirements to be able to use this module, see [Requirements](#) for details.

To use it in a playbook, specify: `community.mongodb.mongodb_user`.

New in community.mongodb 1.0.0

- Synopsis
- Requirements
- Parameters
- Notes
- Examples
- Return Values

Synopsis

- Adds or removes a user from a MongoDB database.

Requirements

The below requirements are needed on the host that executes this module.

- pymongo

Parameters

Parameter	Comments
atlas_auth	Authentication path intended for MongoDB Atlas instances

Parameters	
Parameter	Comments
<code>atlas_auth</code> boolean	Authentication path intended for MongoDB Atlas Instances Choices: <code>false</code> ← (default) <code>true</code>
<code>auth_mechanism</code> % string	Authentication type. Choices: <code>"SCRAM-SHA-256"</code> <code>"SCRAM-SHA-1"</code> <code>"MONGODB-X509"</code> <code>"GSSAPI"</code> <code>"PLAIN"</code>
<code>authentication_restrictions</code> list / elements=any	The authentication restrictions the server enforces on the user. Specifies a list of IP addresses and CIDR ranges the user is allowed to connect from and/or server addresses they can connect to. Provide a list of dictionaries with the following fields: <code>clientSource</code> (list) and <code>serverAddress</code> (list). Provide an empty list if you don't want to use the field. Default: <code>[]</code>
<code>connection_options</code> list / elements=any	Additional connection options. Supply as a list of dicts or strings containing key value pairs separated with '='.
<code>create_for_localhost_exception</code> path	This parameter is only useful for handling special treatment around the localhost exception. If <code>login_user</code> is defined, then the localhost exception is not active and this parameter has no effect. If this file is NOT present (and <code>login_user</code> is not defined), then touch this file after successfully adding the user. If this file is present (and <code>login_user</code> is not defined), then skip this task.
<code>database</code> aliases: db string / required	The name of the database to add/remove the user from.
<code>login_database</code> string	The database where login credentials are stored. Default: <code>"admin"</code>
<code>login_host</code> string	The host running MongoDB instance to login to.

In the list of modules we can also find modules to execute shell commands on remote servers

Index of all Lookup Plugins	ansible.builtin
- Index of all Modules - amazon.aws - ansible.builtin - ansible.mysql - ansible.netcommon - ansible.posix	<code>ansible.builtin.add_host</code> – Add a host (and alternatively a group) to the ansible-playbook in-memory inventory <code>ansible.builtin.apk</code> – Manages apk-packages <code>ansible.builtin.apk_key</code> – Add or remove an apk key DEPRECATED <code>ansible.builtin.apk_repository</code> – Add and remove APT repositories DEPRECATED <code>ansible.builtin.assemble</code> – Assemble configuration files from fragments <code>ansible.builtin.assert</code> – Asserts given expressions are true

-> **ansible.builtin.shell**

Index of all Lookup Plugins	docs.ansible.com/projects/ansible/latest/collections/index_module.html#ansible-builtin
- Channel 4 - BBC iPlayer - Home - Trello - Wise - Login - Google NotebookLM - Overleaf - CV - CodeSignal Learn - Cloud Linode - Linux	
Ansible Community Documentation	
- Index of all Lookup Plugins - Index of all Modules - amazon.aws - ansible.builtin - ansible.mysql - ansible.netcommon - ansible.posix	<code>ansible.builtin.service</code> – Manage services <code>ansible.builtin.service_facts</code> – Return service state information as fact data <code>ansible.builtin.set_fact</code> – Set host variable(s) and fact(s). <code>ansible.builtin.set_stats</code> – Define and display stats for the current ansible run <code>ansible.builtin.setup</code> – Gathers facts about remote hosts <code>ansible.builtin.shell</code> – Execute shell commands on targets <code>ansible.builtin.slurp</code> – Slurps a file from remote nodes <code>ansible.builtin.stat</code> – Retrieve file or file system status

We also have modules to work with directories and files

- cisco.ios.ios_config — push configuration lines (like VLANs, interface settings)
- cisco.ios.ios_command — run show commands and capture output
- cisco.ios.ios_facts — gather device info (version, interfaces, running config)
- Juniper equivalents: junipernetworks.junos.junos_config, junos_facts
- Arista equivalents: arista.eos.eos_config, eos_facts

Example playbook — push a VLAN to a Cisco switch:

- name: Configure VLAN on switch

hosts: switches

gather_facts: no

tasks:

- name: Add VLAN 10

cisco.ios.ios_config:

lines:

- vlan 10
- name Sales

- name: Backup running config

cisco.ios.ios_command:

commands:

- show running-config

register: output

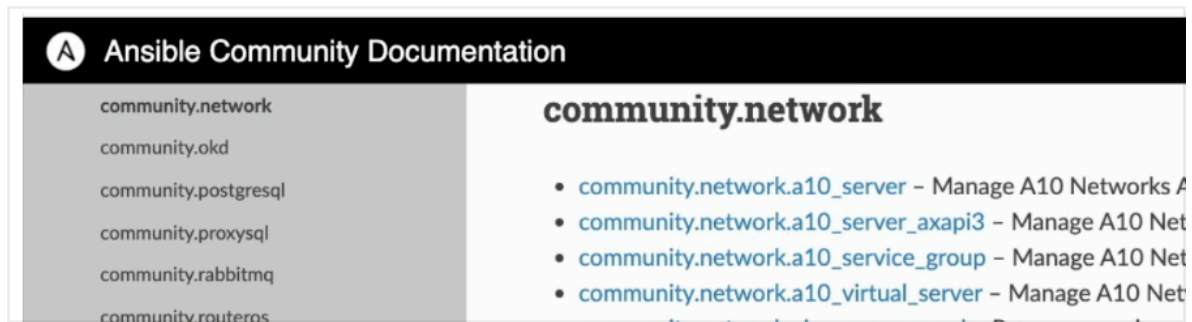
Real-world use cases this unlocks:

- Pushing the same VLAN/ACL/routing config to 200 switches at once, consistently
- Nightly config backups across your whole network estate
- Compliance checks — e.g. verify no device has a default password or an insecure protocol enabled
- Zero-touch provisioning of new switches

So the mental model is the same as with servers (declare desired state, Ansible makes it so, idempotent where the module supports it), just swapping ssh+Python modules for network_cli/netconf+vendor collections underneath.

— end tangent —

Below screenshot from Nana's video of the network module:



Q:

The teacher show in her video community.network from the ansible documentation but I can't find it, i guess the tecaher's video is too old and now there is not community.network. can you tell me the module that replace community.network?

A:

Let me check the current state of this collection to give you an accurate answer rather than going from memory.

- Web Search("community.network ansible collection status 2026 deprecated moved modules")
Did 1 search in 8s

Good news — community.network isn't literally gone, but it did change status, which is probably why you can't find it in the docs the way the video shows it.

What happened:

- As of Ansible 12 (the "batteries included" community package), community.network was removed from the default bundle. It still exists as a standalone collection on Galaxy/GitHub, but it's no longer installed automatically — you'd need ansible-galaxy collection install community.network to get it back.
- More importantly, many of the vendor-specific modules that used to live inside community.network have been migrated out into their own dedicated, vendor-specific collections. For example, all the Mellanox onyx_* modules moved to mellanox.onyx. community.network even left redirect stubs pointing to the new location (being removed in its 2.0.0).

So there isn't one single "replacement module" — it depends on which specific module your teacher's video was showing. The general pattern to follow:

1. Note the module name from the video (e.g. community.network.ios_config, community.network.onyx_vlan, etc.)
2. Check the vendor it targets, and look for that vendor's own collection instead — e.g.:
 - Cisco IOS/NXOS/ASA → cisco.ios, cisco.nxos, cisco.asa
 - Juniper → junipernetworks.junos

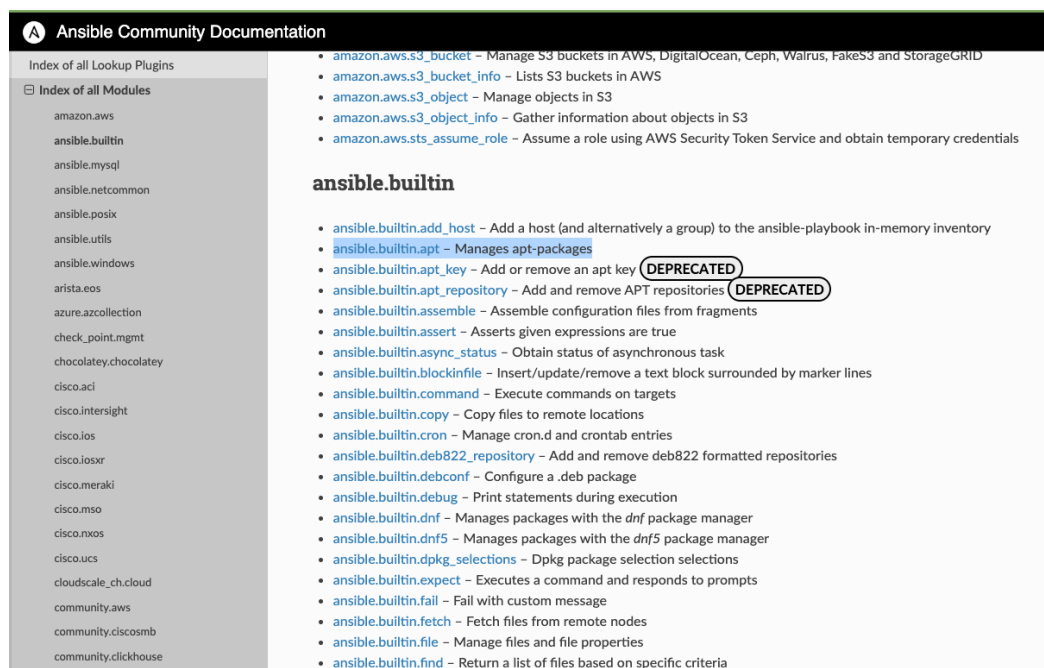
- Arista → arista.eos
- Mellanox → mellanox.onyx
- Dell → dellemc.os10, dellemc.os9, etc.

If you tell me the exact module name from the video, I can point you to its specific new home.

Sources:

- Community.Network — Ansible Community Documentation <https://docs.ansible.com/projects/ansible/latest/collections/community/network/index.html>
- community.network GitHub repository <https://github.com/ansible-collections/community.network>
- Migrating Ansible content to a different collection — Ansible Community Documentation https://docs.ansible.com/projects/ansible/latest/dev_guide/developing_collections_migrating.html

Ansible has a range of modules to install applications like
-> ansible.builtin apt



The screenshot shows the 'Ansible Community Documentation' website. On the left is a sidebar with a search icon and a list of modules under 'Index of all Modules'. The 'ansible.builtin' module is selected. The main content area displays a list of modules under the heading 'ansible.builtin'. The modules listed are:

- amazon.aws.s3_bucket - Manage S3 buckets in AWS, DigitalOcean, Ceph, Walrus, FakeS3 and StorageGRID
- amazon.aws.s3_bucket_info - Lists S3 buckets in AWS
- amazon.aws.s3_object - Manage objects in S3
- amazon.aws.s3_object_info - Gather information about objects in S3
- amazon.aws.sts_assume_role - Assume a role using AWS Security Token Service and obtain temporary credentials
- ansible.builtin - **ansible.builtin**
- ansible.builtin.add_host - Add a host (and alternatively a group) to the ansible-playbook in-memory inventory
- ansible.builtin.apt - **Manages apt-packages**
- ansible.builtin.apt_key - Add or remove an apt key **DEPRECATED**
- ansible.builtin.apt_repository - Add and remove APT repositories **DEPRECATED**
- ansible.builtin.assemble - Assemble configuration files from fragments
- ansible.builtin.assert - Asserts given expressions are true
- ansible.builtin.async_status - Obtain status of asynchronous task
- ansible.builtin.blockinfile - Insert/update/remove a text block surrounded by marker lines
- ansible.builtin.command - Execute commands on targets
- ansible.builtin.copy - Copy files to remote locations
- ansible.builtin.cron - Manage cron.d and crontab entries
- ansible.builtin.deb822_repository - Add and remove deb822 formatted repositories
- ansible.builtin.debconf - Configure a .deb package
- ansible.builtin.debug - Print statements during execution
- ansible.builtin.dnf - Manages packages with the dnf package manager
- ansible.builtin.dnf5 - Manages packages with the dnf5 package manager
- ansible.builtin.dpkg_selections - Dpkg package selection selections
- ansible.builtin.expect - Executes a command and responds to prompts
- ansible.builtin.fail - Fail with custom message
- ansible.builtin.fetch - Fetch files from remote nodes
- ansible.builtin.file - Manage files and file properties
- ansible.builtin.find - Return a list of files based on specific criteria

Because entailing applications is a very use case for ansible.

Here is another one yum which is now dnf

- **ansible.builtin.wait_for_connection** - **Waits until remote system is reachable**
- **ansible.builtin.yum_repository** - **Add or remove YUM repositories**

yum (Yellowdog Updater, Modified) is the package manager for RHEL-family

Linux distributions — specifically older/legacy versions:

- RHEL (Red Hat Enterprise Linux) up through version 7
- CentOS up through version 7
- Fedora (older versions, before it switched to dnf)

Its successor, dnf, is the default on RHEL 8+, CentOS 8+, Fedora, Rocky Linux, and AlmaLinux — dnf is a rewrite of yum with better dependency resolution, but the command syntax is largely compatible (dnf install works similarly to yum install).

In Ansible, the yum module still works on newer systems too (it transparently calls dnf under the hood on RHEL 8+), but the dnf module is the more "native" choice there.

COLLECTION

Each of the modules we see are part of a COLLECTION

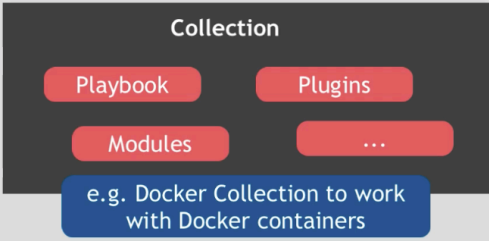


A collection bundles or groups Ansible content.

The content can be a playbook, plugins, modules, etc

What is a Collection?

- ▶ A packaging format for bundling and distributing Ansible content
- ▶ Can be released and installed independent of other collections



The diagram shows a dark grey box labeled 'Collection' at the top. Inside the box are four red rounded rectangles: 'Playbook', 'Plugins', 'Modules', and '...'. Below the box is a blue rounded rectangle with the text 'e.g. Docker Collection to work with Docker containers'.

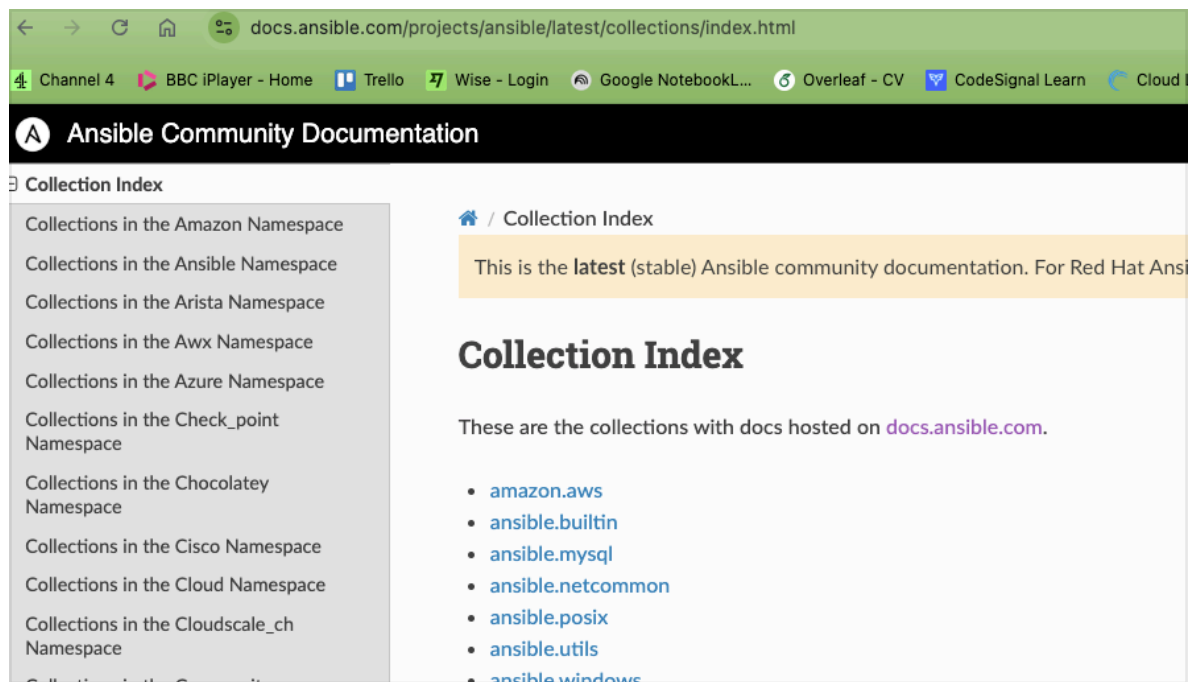


▶ A single bundle

Before we saw the list of 'Indexes of all modules and plugins'
Now for collections we have 'Collection Index'

Ansible Community Documents

- Network Advanced Topics
- Network Developer Guide
- ANSIBLE GALAXY**
- Galaxy User Guide
- Galaxy Developer Guide
- REFERENCE & APPENDICES**
- Collection Index**
-  Indexes of all modules and plugins
- Playbook Keywords

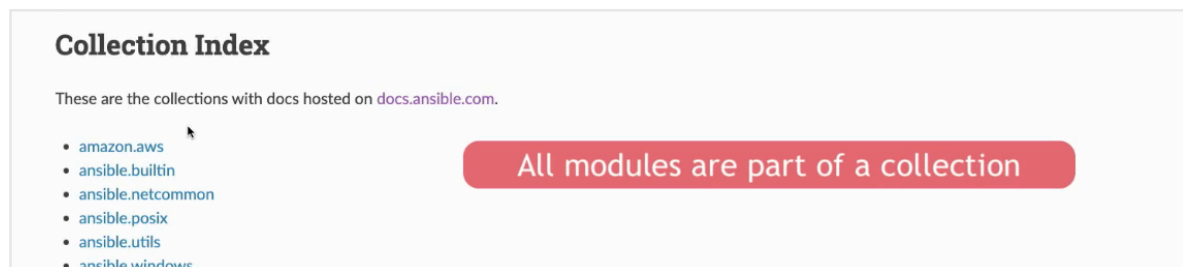


So instead of having modules separately or plugins separately a Collections bundles all of them together and package them in a single bundle.

And the bundle is a collection

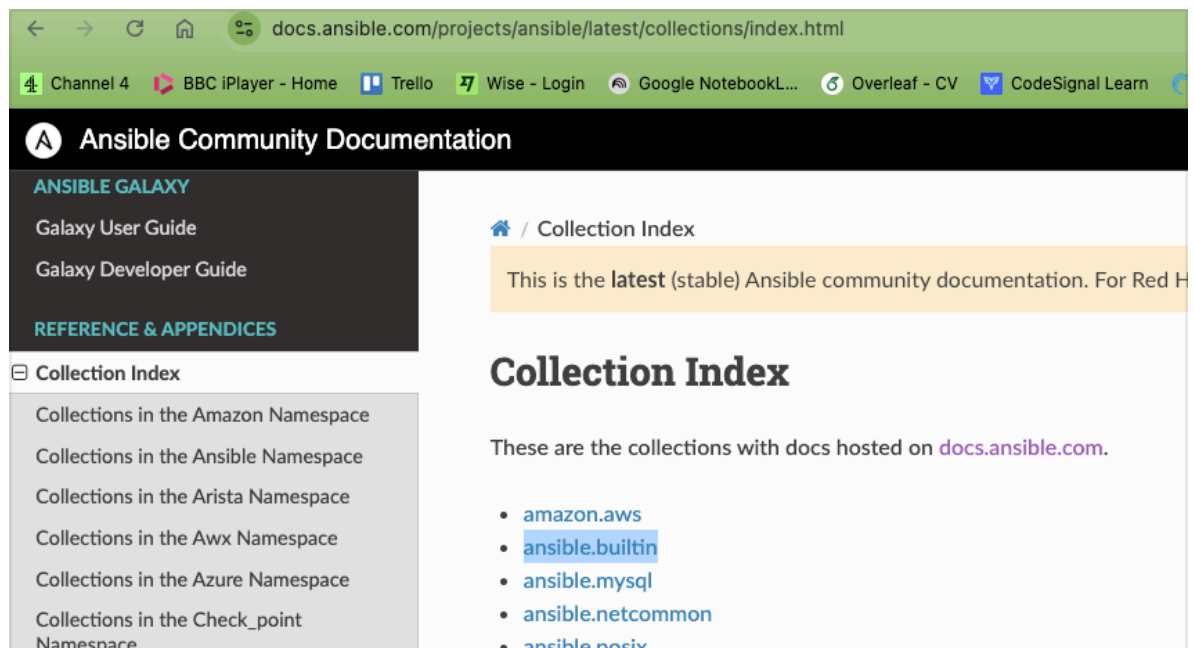
A collection is easier to distribute and share all the ansible content inside.

So all the modules are part of a collection:

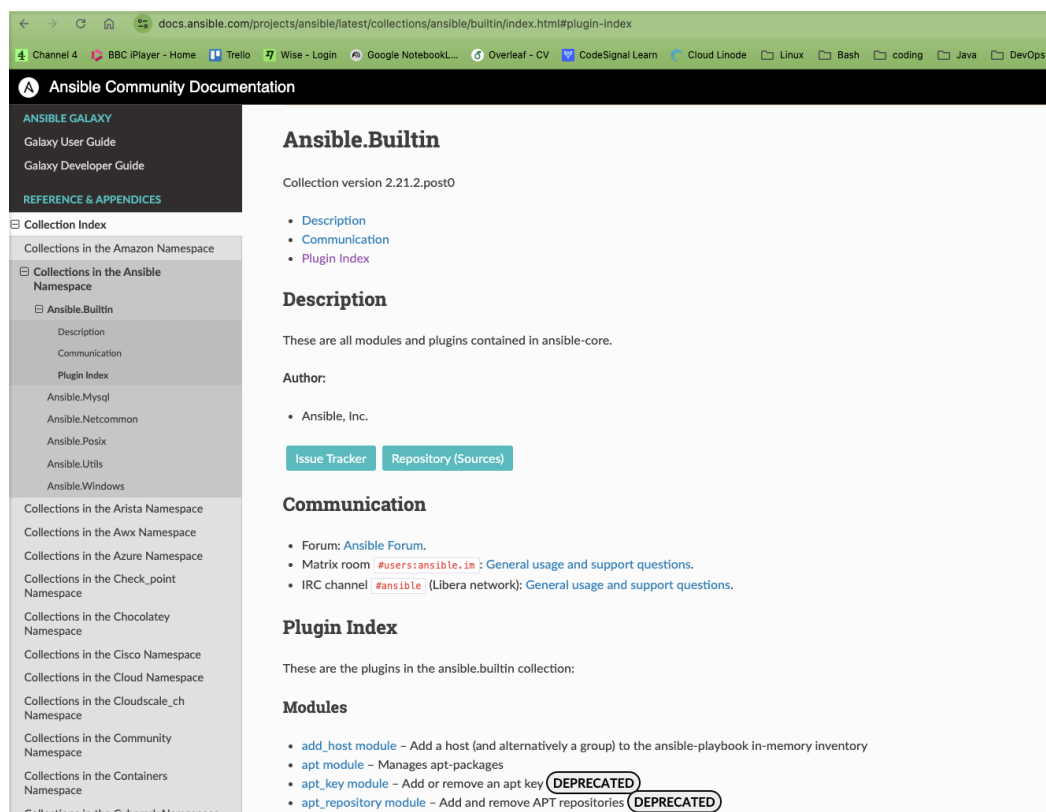


And above we can see the list of the collections available in Ansible.

So the modules we used in Module 15 - chapter 7 which were 'apt' and 'service' are part of the same collection 'ansible.builtin'



And we can see what the collection contents are:



So we can see it contains modules and plugins

Plugins -> Ansible code that adds to Ansible's functionality or modules.

Cache Plugins

- [jsonfile cache](#) – JSON formatted files.
- [memory cache](#) – RAM backed, non persistent

Callback Plugins

- [default callback](#) – default Ansible screen output
- [junit callback](#) – write playbook output to a JUnit file.
- [minimal callback](#) – minimal Ansible screen output
- [oneline callback](#) – oneline Ansible screen output
- [tree callback](#) – Save host events to files

Connection Plugins

- [local connection](#) – execute on controller
- [paramiko_ssh connection](#) – Run tasks via Python SSH (paramiko)
- [psrp connection](#) – Run tasks over Microsoft PowerShell Remoting Protocol
- [ssh connection](#) – connect via SSH client binary
- [winrm connection](#) – Run tasks over Microsoft's WinRM

Filter Plugins

Ansible Plugins

- ▶ Pieces of code that add to Ansible's functionality or modules
- ▶ You can also write your own Plugins

Let's say we have a module for AWS EC2 that creates/terminates EC2 instances in AWS, then with a plugin we can enhance that module's functionality like adding a feature that filters which EC2 instance we want to terminate.

We can also see the modules one of which is the apt module that we used:

A Ansible Community Documentation

- Collections in the Azure Namespace
- Collections in the Check_point Namespace
- Collections in the Chocolatey Namespace
- Collections in the Cisco Namespace
- Collections in the Cloud Namespace
- Collections in the Cloudscale_ch Namespace
- Collections in the Community Namespace
- Collections in the Containers Namespace
- Collections in the Cyberark Namespace
- Collections in the Dellemc Namespace

- Forum: [Ansible Forum](#).
- Matrix room [#users:ansible.im](#) : [Generate](#)
- IRC channel [#ansible](#) (Libera network)

Plugin Index

These are the plugins in the ansible.builtins namespace

Modules 🔗

- [add_host module](#) – Add a host (and alt
- [apt module](#) – Manages apt-packages
- [apt_key module](#) – Add or remove an ap
- [apt_repository module](#) – Add and remc
- [assemble module](#) – Assemble configur
- [assert module](#) – Asserts given expressi

Ansible Community Documentation

Ansible

Search docs

ANSIBLE GETTING STARTED

Getting started with Ansible

Getting started with Execution Environments

INSTALLATION, UPGRADE & CONFIGURATION

Installation Guide

Ansible Porting Guides

USING ANSIBLE

Building Ansible inventories

Using Ansible command line tools

Using Ansible playbooks

Protecting sensitive data with Ansible vault

Using Ansible modules and plugins

Using Ansible collections

Using Ansible on Windows, BSD, and z/OS UNIX

Ansible tips and tricks

CONTRIBUTING TO ANSIBLE

Ansible Community Guide

Ansible Collections Contributor Guide

ansible-core Contributors Guide

Advanced Contributor Guide

Ansible documentation style guide

EXTENDING ANSIBLE

Developer Guide

ansible.builtin.apt module – Manages apt-packages

Note

This module is part of [ansible-core](#) and included in all Ansible installations. In most cases, you can use the short module name `apt` even with you use the Fully Qualified Collection Name (FQCN) `ansible.builtin.apt` for easy linking to the module documentation and to avoid conflicts.

- Synopsis
- Requirements
- Parameters
- Attributes
- Notes
- See Also
- Examples
- Return Values

Synopsis

- Manages `apt` packages (such as for Debian/Ubuntu).

Requirements

The below requirements are needed on the host that executes this module.

- python3-apt
- aptitude (before 2.4)

Parameters

Parameter	Comments
<code>allow_change_held_packages</code> boolean added in ansible-core 2.13	Allows changing the version of a package which is on the apt hold list. Choices: <code>false</code> ← (default) <code>true</code>

We also saw collections that have the word ‘community.’ -> they are collections maintained by the Ansible community

- community.aws
- community.azure
- community.ciscosmb
- community.crypto
- community.digitalocean
- community.dns
- community.docker
- community.general
- community.grafana
- community.hashi_vault
- community.hrobot
- community.libvirt
- community.mongodb
- community.mysql
- community.network
- community.okd
- community.postgresql
- community.proxysql
- community.rabbitmq
- community.routeros
- community.sap
- community.sap_libs
- community.sops
- community.vmware
- community.windows

Maintenance of Modules

Maintenance

Collection	Code location	Maintained by
ansible.builtin	ansible/ansible repo on GitHub	core team
distributed on Galaxy	various; follow repo link	community or partners
distributed on Automation Hub	various; follow repo link	content team or partners

These collections are available when we installed ansible.

To test, let's run:

-> ansible-galaxy collection list

The name ‘ansible-galaxy’: Ansible itself is named after a fictional device from Ursula K. Le Guin's sci-fi novels that allows instant communication across any distance — fitting for a tool that instantly configures machines anywhere. Galaxy continues that space theme: it's the public hub (galaxy.ansible.com) where the community shares Ansible content, the way npm is for Node packages or PyPI is for Python packages.

Why it exists / what problem it solves:

Without it, you'd have to write every role and module yourself from scratch — e.g., a full role to install and configure nginx, or all those Cisco/Juniper network modules we discussed earlier. `ansible-galaxy` gives you:

1. A registry to pull pre-built content from — roles and collections written by Red Hat, vendors (Cisco, Juniper, Arista), and the community, so you don't reinvent things like "configure a Cisco switch" or "set up PostgreSQL."
2. A dependency/install mechanism — `ansible-galaxy collection install cisco.ios` or `ansible-galaxy install -r requirements.yml` pulls exact versions your playbook needs, similar to `pip install -r requirements.txt`.
3. A way to package and share your own roles/collections — `ansible-galaxy init` scaffolds a new role/collection with the standard directory structure, so teams share reusable automation consistently.
4. The `list/info` subcommands (like we just covered) to inspect what's installed locally.

So practically: it's Ansible's package manager — it's why you can `pip install cisco.ios` (well, `ansible-galaxy collection install cisco.ios`) instead of hand-writing every network module yourself.

Ansible is the actual name of the automation tool itself (what runs playbooks, connects to hosts, executes modules). That's the core product.

ansible-galaxy is just one command-line tool that ships with Ansible — specifically the one that manages roles/collections (`install`, `list`, `init`, etc.). Think of it the same way `git` has subcommands like `git commit`. `ansible` and `ansible-galaxy` are siblings, both part of the same package, but separate executables with separate jobs:

Command	Purpose
<code>ansible</code>	Run a single ad-hoc task against hosts
<code>ansible-playbook</code>	Run a playbook (what you've been doing with <code>my-playbook.yml</code>)
<code>ansible-galaxy</code>	Install/manage roles and collections
<code>ansible-vault</code>	Encrypt secrets
<code>ansible-doc</code>	Show documentation for a module

So "Galaxy" is not the name of the whole tool — **it's the name of the content-sharing hub/registry** (`galaxy.ansible.com`), and **ansible-galaxy** is the CLI that talks to it.

-> Same relationship as: Docker is the tool, Docker Hub is the registry, and docker pull is the command that talks to it.

So the general pattern is:

-> ansible-galaxy <category> <action> [target] [options]

Part	What it means
ansible-galaxy	The executable/program you're calling – the CLI tool bundled with Ansible that manages roles and collections (installing, listing, creating them).
collection	The subcommand category – tells ansible-galaxy you want to work with <i>collections</i> specifically (as opposed to the older role category, e.g. ansible-galaxy role list, which manages standalone roles instead).
list	The action to perform within that category – "show me what's installed." Other actions you could put here instead: install (ansible-galaxy collection install cisco.ios), init (scaffold a new collection), or verify (check installed content matches what was published).

And I execute the command

```
~ (-zsh) #1 * Explain ps aux command (claude)
> ansible-galaxy collection list
[WARNING]: Collection at '/usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible/_internal/ansible_collections/ansible/_protomatter' does not have a MANIFEST.json file, nor has it galaxy.yml: cannot detect version.

# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible_collections
Collection      Version
-----
ansible._protomatter *

# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible_collections
Collection      Version
-----
amazon.aws      11.3.0
ansible.mysql   5.0.1
ansible.netcommon 8.5.3
ansible.posix    2.2.0
ansible.utils    6.0.3
ansible.windows 3.6.1
arista.eos       12.1.2
azure.azcollection 3.19.0
check_point.mgmt 6.9.0
chocolatey.chocolatey 1.6.0
cisco.aci        2.13.0
cisco.intersight 2.19.0
cisco.ios        11.4.2
cisco.iosxr      12.3.2
cisco.meraki     2.24.0
cisco.mso        2.13.0
cisco.nxos       11.2.0
cisco.ucs        1.16.0
cloudscale_ch.cloud 2.5.3
community.aws    11.0.0
community.ciscosmb 1.0.12
community.clickhouse 2.2.0
community.crypto 3.2.2
community.dns    4.0.1
community.docker 5.2.1
```

And now we see the list of collections located in the path

-> '/usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible/_internal/ansible_collections'

```
# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible/_internal/ansible_collections
Collection      Version
-----
ansible._protomatter *

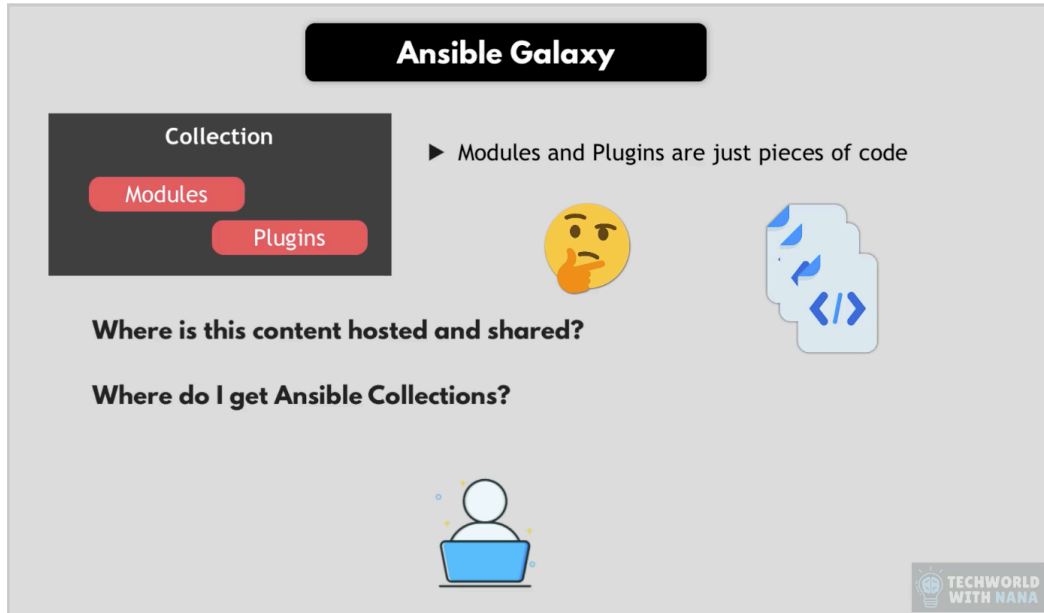
# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible_collections
Collection      Version
-----
amazon.aws      11.3.0
ansible.mysql   5.0.1
ansible.netcommon 8.5.3
ansible.posix    2.2.0
```

Ansible Galaxy

Collections groups Ansible content -> modules + plugins

modules + plugins are code

And code has to be somewhere!



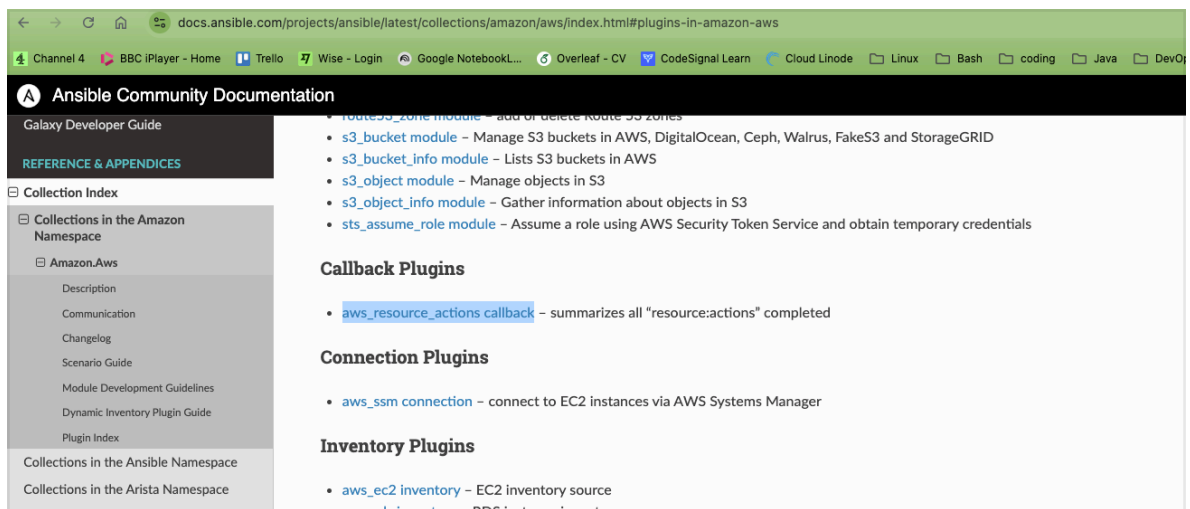
So we can download the code locally.

One of the main **hubs** of collections is Ansible Galaxy

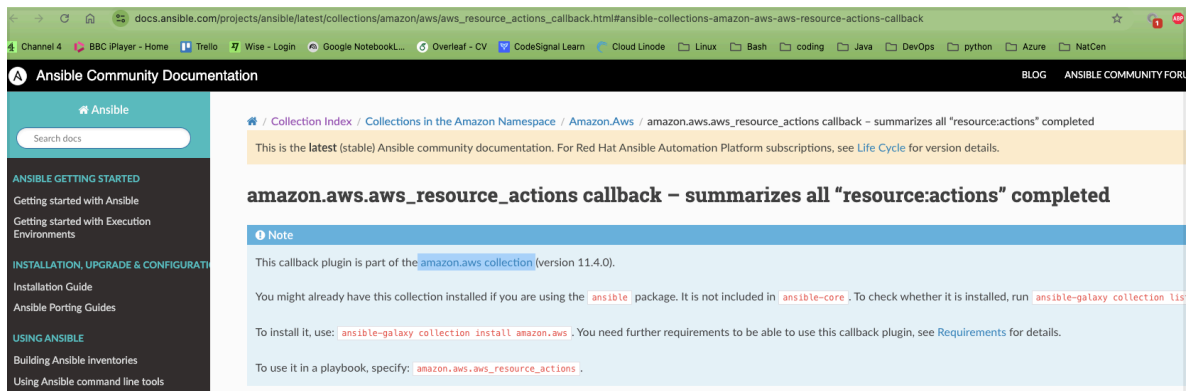


For example we want to check the code of an AWS plugin

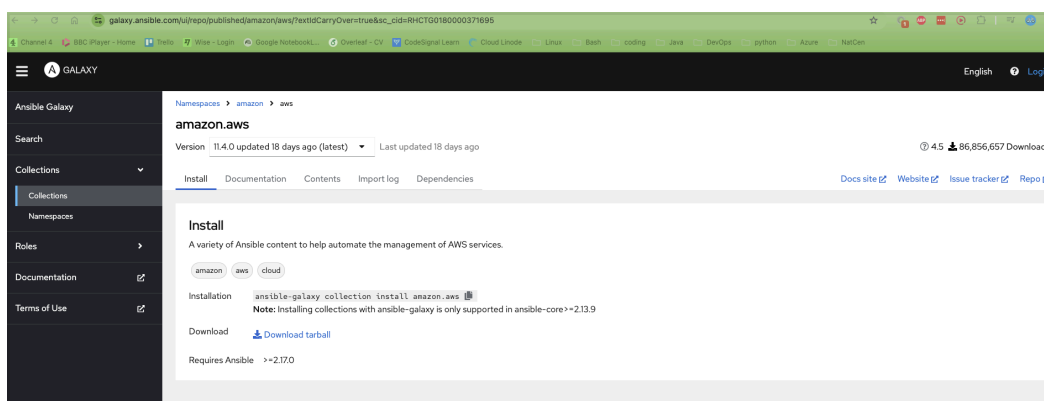
-> `aws_resource_actions callback`

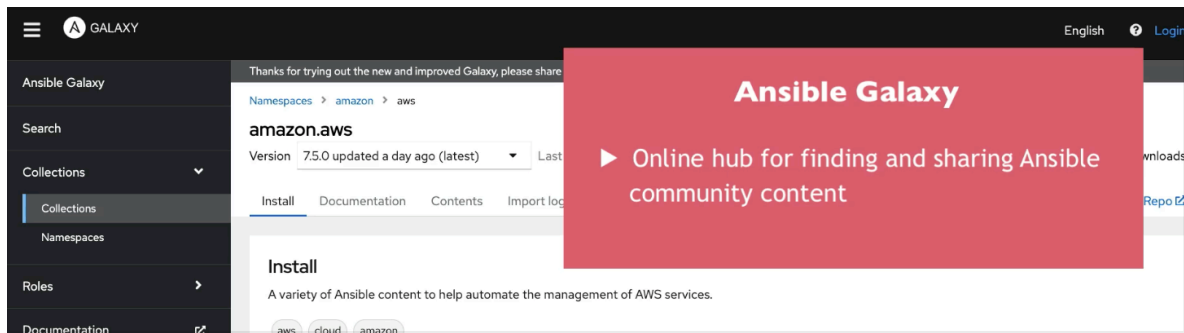


We will be directed to

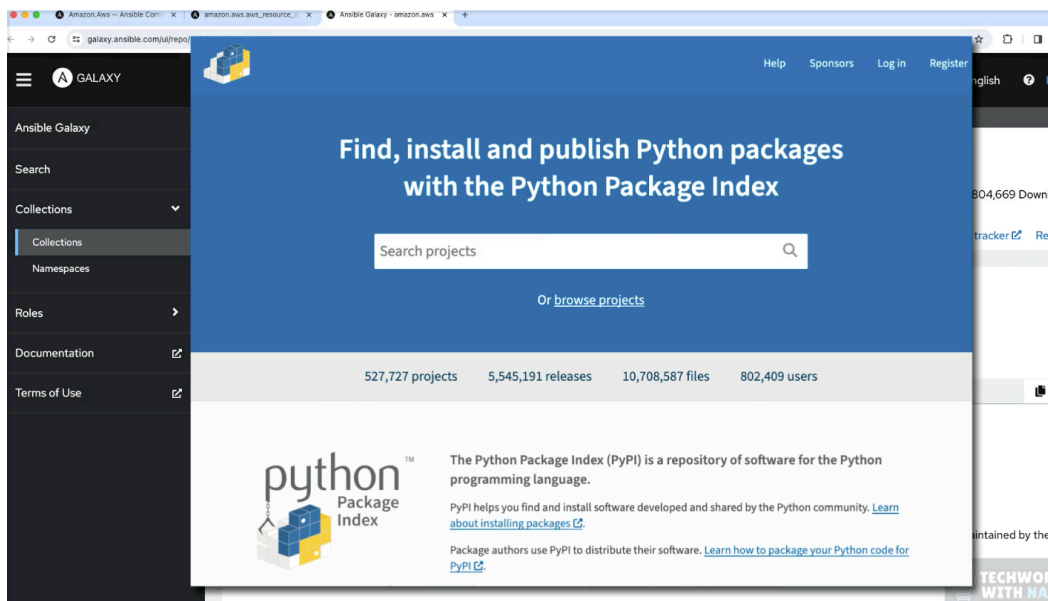


And then we click on **amazon.aws collection** and we are directed to Ansible Galaxy where the code can be downloaded

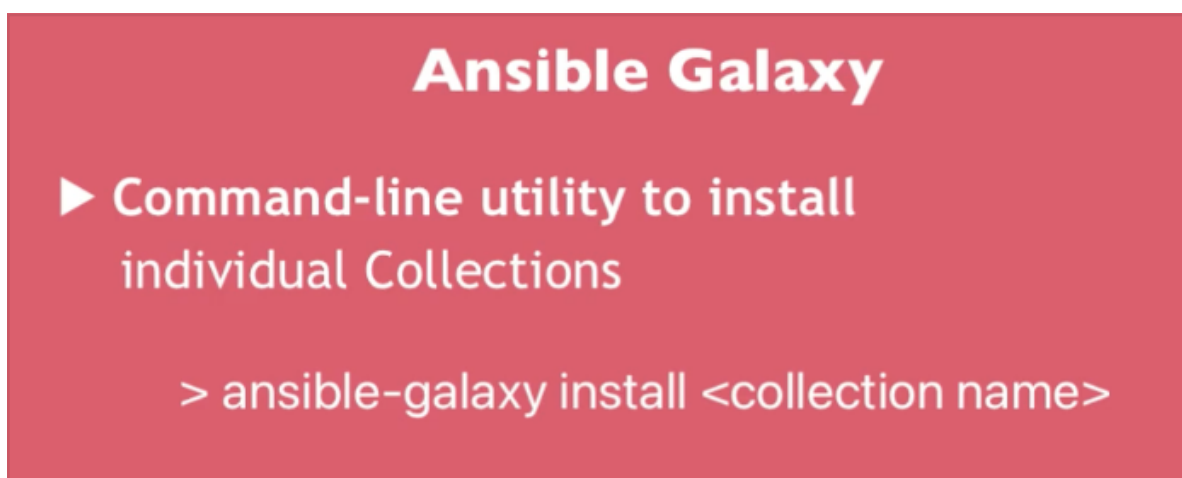




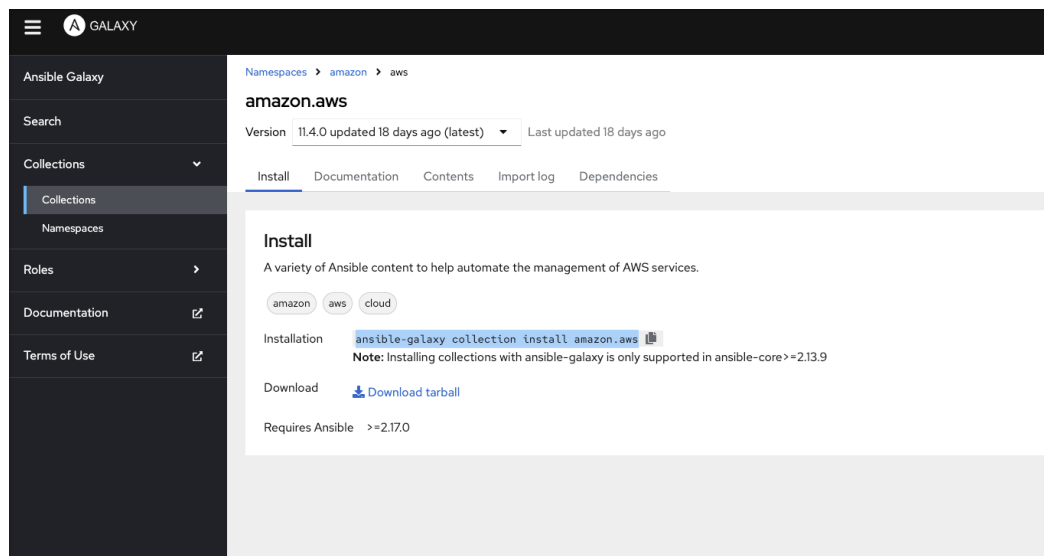
So ansible galaxy is a repository like pypi or terraform registry.



And ansible-galaxy is the command line that fetches the collection from the repo



So when we need to install a collection we execute the command line provided by ansible galaxy like in the example below
-> ansible-galaxy collection install amazon.aws



Q:

When I installed ansible I got installed many collection, I guess all the ones from ansible galaxy? but other collections are not installed?

A:

Good question — let's clarify the distinction, because it's easy to conflate "installed on my machine" with "everything on Galaxy."

Ansible Galaxy (galaxy.ansible.com) is the whole catalog — thousands of collections published by Red Hat, vendors, and the community. Nowhere close to all of them are on your machine.

What actually gets installed depends on how you installed Ansible:

1. `pip install ansible` (the full "community package") — this bundles a curated subset of collections that Red Hat/the community decided to ship by default, so common things (community.general, ansible.posix, cisco.ios, etc.) work out of the box without extra steps. This is likely why your ansible-galaxy collection list shows "many" — you got the batteries-included package, not a bare install.
2. `pip install ansible-core` (the minimal engine) — this ships with zero collections, just the core engine and a handful of built-in modules. You'd have to install every collection yourself.

Either way, anything not in that bundled set is not installed — you have to explicitly pull it:

```
ansible-galaxy collection install <namespace.collection>
```

This is exactly the situation you hit with community.network: it used to be part of the default bundled set, got dropped from it in Ansible 12, and now you have to install it manually if you want it — it still exists on Galaxy, it's just no longer pre-installed for you.

So: your machine has the bundled subset that ships with whatever ansible package version you installed, not "everything from Galaxy." Anything beyond that list requires an explicit install.

Upgrade collection

I executed -> pip install ansible so I got the full "community package" and If I want o upgrade a collection that is available in my ansible then I just pick the one I want to upgrade.

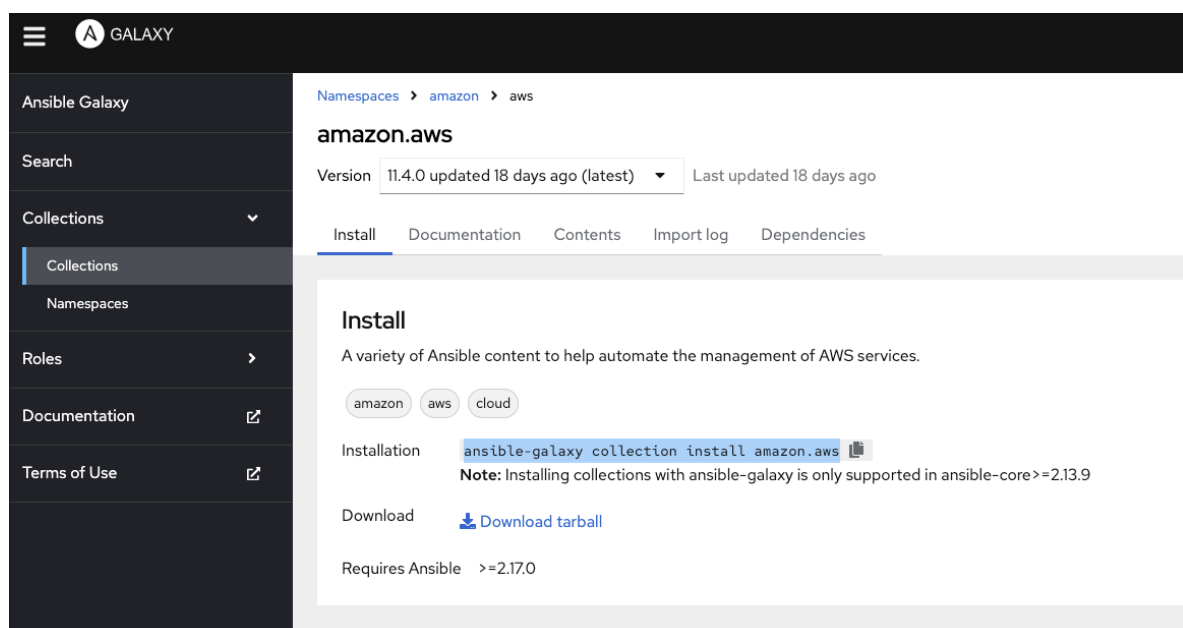
For example 'amazon.aws' collection

```
> ansible-galaxy collection list
[WARNING]: Collection at '/usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.7/site-packages/ansible/_protomatter' does not have a MANIFEST.json

# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.7/site-packages/ansible/_protomatter
Collection                                     Version
-----
ansible._protomatter                         *

# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.7/site-packages/ansible/collections
Collection                                     Version
-----
amazon.aws                                   11.3.0
```

So I take the command line from



The screenshot shows the Ansible Galaxy website interface. On the left is a dark sidebar with a menu icon and the 'GALAXY' logo. Below the logo are links for 'Ansible Galaxy', 'Search', 'Collections' (with a dropdown arrow), 'Namespaces', 'Roles' (with a right arrow), 'Documentation' (with an external link icon), and 'Terms of Use' (with an external link icon). The main content area has a breadcrumb trail 'Namespaces > amazon > aws'. Below this is the title 'amazon.aws' and a version selector showing '11.4.0 updated 18 days ago (latest)' with a dropdown arrow. To the right of the version is the text 'Last updated 18 days ago'. Below the version selector are tabs for 'Install', 'Documentation', 'Contents', 'Import log', and 'Dependencies'. The 'Install' tab is selected and shows the heading 'Install' followed by the description 'A variety of Ansible content to help automate the management of AWS services.' Below the description are three tags: 'amazon', 'aws', and 'cloud'. The 'Installation' section shows the command 'ansible-galaxy collection install amazon.aws' with a copy icon. Below the command is a note: 'Note: Installing collections with ansible-galaxy is only supported in ansible-core>=2.13.9'. The 'Download' section has a link 'Download tarball' with a download icon. At the bottom, it says 'Requires Ansible >=2.17.0'.

And I execute

-> ansible-galaxy collection install amazon.aws --upgrade

```
> ansible-galaxy collection install amazon.aws --upgrade
Starting galaxy collection install process
Process install dependency map
Starting collection install process
Downloading https://galaxy.ansible.com/api/v3/plugin/ansible/content/published/collections/artifacts/amazon-aws-11.4.0.tar.gz to
/Users/astrid/.ansible/tmp/ansible-local-14525id7nfyec/tmp9h6d6sl9/amazon-aws-11.4.0-jtepo5a6
Installing 'amazon.aws:11.4.0' to '/Users/astrid/.ansible/collections/ansible_collections/amazon/aws'
amazon.aws:11.4.0 was installed successfully
```

And now we check the list again

```
> ansible-galaxy collection list
[WARNING]: Collection at '/usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible/_internal/ansible_collections/ansible/_protomatter' does not have a MANIFEST.json file, nor has it galaxy.yml: cannot detect version.

# /Users/astrid/.ansible/collections/ansible_collections
Collection                                     Version
-----
amazon.aws                                   11.4.0

# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible/_internal/ansible_collections
Collection                                     Version
-----
ansible._protomatter                         *

# /usr/local/Cellar/ansible/14.1.0/libexec/lib/python3.14/site-packages/ansible_collections
Collection                                     Version
-----
amazon.aws                                   11.3.0
ansible.mysql                               5.0.1
ansible.netcommon                           8.5.2
```

We see we have the collection twice one in the original path and the upgrade is installed in /Users/astrid path and it has a recent version.

I now have the same collection in two different search locations because of where the install command defaults to writing.

What happened:

- /usr/local/Cellar/ansible/14.1.0/.../site-packages/ansible_collections — this is the copy that shipped bundled with Ansible itself when you installed it via Homebrew (version 11.3.0, whatever was current when that Ansible release was packaged).
- /Users/astrid/.ansible/collections/ansible_collections — this is your user-level install location, which is where ansible-galaxy collection install writes by default when you don't pass a -p/path flag.

So when you ran the upgrade, Ansible didn't touch (and generally shouldn't touch) the bundled copy inside the Homebrew Cellar — that's managed by the package manager, not by you. Instead, it installed the newer 11.4.0 into your user directory as a separate copy.

Does this cause a conflict? No — because of search order. Ansible checks collection paths in priority order, and your user directory (~/.ansible/collections) is checked before the one bundled inside the Ansible package installation. So:

- Your playbooks will use 11.4.0 (the user-level one wins)
- The 11.3.0 bundled copy just sits there unused, shadowed by the newer one

Ansible has a defined order for which one "wins" when there's overlap. You don't need to delete anything; it's working as intended.

If you ever want to confirm which one is actually in effect for a given collection, that's what `ansible-galaxy collection list amazon.aws` (narrowed to one collection) or checking the very first path listed for it is for — first match wins.

Q:

wouldn't this be in a way a waste of memory? I mean if I upgrade the collection why to keep the original one?

A:

Fair concern, but in practice it's not meaningful waste — for two reasons:

1. Size is negligible. Ansible collections are just Python/YAML text files (modules, docs, plugins) — `amazon.aws` is maybe a few MB. Keeping an old copy alongside a new one costs kilobytes to low single-digit MB, not something worth worrying about on modern disks.

2. They serve different purposes, so it's not really "duplication" in a wasteful sense:

- The bundled copy (inside the Homebrew Cellar) is owned by the package manager, not you. It's the version that was tested and shipped together with that specific Ansible release — a "known-good fallback." Homebrew tracks it as part of the ansible formula's file manifest.
- The user copy (`~/.ansible/collections`) is your override, letting you get a newer collection version without waiting for Homebrew to ship a new Ansible release with it bundled.

If you manually deleted the bundled one to "save space," a few things bite you:

- brew would notice its tracked files are missing/modified and could complain on the next brew doctor/upgrade/reinstall of ansible.
- The next time you brew upgrade ansible, it gets recreated anyway (fresh Cellar directory for the new version) — so deleting it now is a one-time, low-value cleanup at best.

If you want a cleaner setup going forward, the real fix isn't deleting the bundled copy — it's picking one consistent strategy:

- Either always manage collections yourself via `ansible-galaxy collection install` (accept that your user copy will shadow/duplicate the bundled one, and don't worry about it), or
- Never manage them yourself, and just get whatever version ships with each

brew upgrade ansible (simpler, but you're at the mercy of Homebrew's release cadence for collection versions).

Given you already upgraded manually, you're implicitly in the first camp — which is completely normal and how most people who need a newer collection version than their OS package provides end up working.

Q:

And if I upgrade again then this case is different and I won't end up with 3 copies. the one in my ~/.ansible/collections) will be upgraded and I will have only one in that path

A:

Correct — you'll still only have two paths total (the Homebrew bundled one + your user one), and the user path stays a single copy, just updated in place.

Why: ansible-galaxy collection install (with --upgrade, or just re-running install for a newer version) treats each path as holding one version per collection. When it installs into ~/.ansible/collections/ansible_collections/amazon/aws/, a new install to that same path overwrites the directory contents — it doesn't version-suffix it or keep the old files around. So going from 11.4.0 → 11.5.0 (say) replaces the files in place; you won't accumulate 11.4.0 and 11.5.0 side by side in the same path.

So your end state after another upgrade:

~/.ansible/collections/.../amazon.aws → 11.5.0 (overwritten, single copy)
.../Cellar/ansible/14.1.0/.../amazon.aws → 11.3.0 (untouched, still the bundled one)

Still two paths, still two versions visible in ansible-galaxy collection list — but that second one only changes if Homebrew ships a new Ansible version with a newer bundled amazon.aws, which is a separate event from your own ansible-galaxy upgrades.

Create own collection

If the ansible project is too big

► For bigger Ansible projects



Plugins



Modules



Playbooks

There is a standard structure to create collections

Create own Collection

- For bigger Ansible projects
- Collections follow a simple data structure
 - Required:
a **galaxy.yml** file (containing metadata)
at the root level of the collection

```
collection/  
├── docs/  
├── galaxy.yml  
├── meta/  
│   └── runtime.yml  
├── plugins/  
│   ├── modules/  
│   │   └── module1.py  
│   ├── inventory/  
│   └── .../  
├── README.md  
├── roles/  
│   ├── role1/  
│   ├── role2/  
│   └── .../  
├── playbooks/  
│   ├── files/  
│   ├── vars/  
│   ├── templates/  
│   └── tasks/  
└── tests/
```



Ansible Namespaces

Structure of the modules names

The screenshot shows the Ansible Community Documentation website. The main content area displays the title "community.docker.docker_image module – Manage docker images". Below the title, a red box contains the text "<namespace>.<collection>.<module name>". Arrows point from the words "community", "docker", and "docker_image" in the title to the corresponding parts of the red box. The left sidebar shows navigation links for "ANSIBLE GETTING STARTED", "INSTALLATION, UPGRADE & CONFIGURATION", and "ANSIBLE PORTING GUIDES". The top navigation bar includes links for "ANSIBLEFEST", "PRODUCTS", "COMMUNITY", and "WEBINARS & TRAINING".

So every collection belongs to a name space.

We can see it here as well, where namespace is 'ansible'

Collection Index

These are the collections with docs

- [amazon.aws](#)
- [ansible.builtin](#)
- [ansible.netcommon](#)
- [ansible.posix](#)

And if we take a look at `community.docker.docker_image` module and look at an example

Examples

```
- name: Pull an image
  community.docker.docker_image:
    name: pacur/centos-7
    source: pull
    # Select platform for pulling. If not specified, will pull whatever docker prefers.
    pull:
      platform: amd64

- name: Tag and push to docker hub
  community.docker.docker_image:
    name: pacur/centos-7:56
    repository: dcopenhagen/myimage:7.56
    push: true
    source: local

- name: Tag and push to local registry
```

Fully Qualified Collection Name (FQCN)

FQCN = to specify the exact source of a module/plugin/...

We see the fully qualified collection name -> FQCN

In Module 15 - chapter 7 we didn't use the FQCN, just 'apt' and just 'service'

```
my-playbook.yaml x
1  ---
2  - name: Configure nginx web server
3    hosts: webserver
4    tasks:
5      - name: uninstall nginx server
6        apt: Caballero-Pardo, Today • Creat
7          name: nginx=1.24.*
8          state: absent
9      - name: stop nginx server
10     service:
11       name: nginx
12       state: stopped
```

That was because `ansible.builtin` is the default namespace and collection name so we didn't need to use them:

```
- name: Install apache httpd (state=present is optional)
  ansible.builtin.apt:
    name: apache2
    state: present

- name: Update repositories cache and install "foo" package
  ansible.builtin.apt:
    name: foo
    update_cache: yes

- name: Remove "foo" package
  ansible.builtin.apt:
    name: foo
```

ansible.builtin = default namespace and collection name

For ansible version 2.10 the developers made sure ansible was backwards compatible so it can handle not using the FQCN as the previous versions (2.9 and earlier) didn't require the FQCN. So for example for the `docker_image` module we can just type
-> `docker_image`

But is good to start using the new structure and use the FQCN

Examples

```
- name: Pull an image
community.docker.docker_image:
  name: pacur/centos-7
  source: pull
  # Select platform for pulling. If not specified, will pull whatever docker prefers.
  pull:
    platform: amd64

- name: Tag and push to docker hub
community.docker.docker_image:
  name: pacur/centos-7:56
  repository: dcopenhagen/myimage:7.56
  push: true
  source: local

- name: Tag and push to local registry
community.docker.docker_image:
  # Image will be centos:7
  name: centos
  # Will be pushed to localhost:5000/centos:7
  repository: localhost:5000/centos
  tag: 7
  push: true
  source: local

- name: Add tag latest to image
community.docker.docker_image:
  name: myimage:7.1.2
  repository: myimage:latest
  # As 'latest' usually already is present, we need to enable overwriting of existing tags:
```



Start using the fully qualified name

e.g. the backwards compatibility may be removed in future versions



As ansible might remove the backwards compatibility in the future.